

UNIT_4 LAB EXERCISE

NAME: Ranjitha.R

REG NO: 192421285

1. You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes dynamic programming to solve the Dice Throw Problem.

```
1 def dice_ways(num_dice, num_sides, target):
2     dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
3     dp[0][0] = 1
4
5     for d in range(1, num_dice + 1):
6         for s in range(1, target + 1):
7             for face in range(1, num_sides + 1):
8                 if s >= face:
9                     dp[d][s] += dp[d - 1][s - face]
10
11     return dp[num_dice][target]
12
13
14 print("Test Case 1:", dice_ways(2, 6, 7))
15 print("Test Case 2:", dice_ways(3, 4, 10))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.856 Mb

Your Output

Test Case 1: 6
Test Case 2: 6

2. In a factory, there are two assembly lines, each with n stations. Each station performs a specific task and takes a certain amount of time to complete. The task must go through each station in order, and there is also a transfer time for switching from one line to another. Given the time taken at each station on both lines and the transfer time between the lines, the goal is to find the minimum time required to process a product from start to end.

```
1 def assembly_line(a1, a2, t1, t2, e1, e2, x1, x2):
2     n = len(a1)
3
4     dp1 = [0] * n
5     dp2 = [0] * n
6
7     dp1[0] = e1 + a1[0]
8     dp2[0] = e2 + a2[0]
9
10    for i in range(1, n):
11        dp1[i] = min(dp1[i - 1] + a1[i],
12                    dp2[i - 1] + t2[i - 1] + a1[i])
13
14        dp2[i] = min(dp2[i - 1] + a2[i],
15                    dp1[i - 1] + t1[i - 1] + a2[i])
16
17    return min(dp1[-1] + x1, dp2[-1] + x2)
18
19
20 a1 = [4, 5, 3, 2]
21 a2 = [2, 10, 1, 4]
22
23 t1 = [7, 4, 5]
24 t2 = [9, 2, 8]
25
26 e1, e2 = 10, 12
27 x1, x2 = 18, 7
28
29 print("Minimum time:", assembly_line(a1, a2, t1, t2, e1, e2, x1, x2))
```

Ctrl+ / Enter here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.976 Mb

Your Output

Minimum time: 35

3. An automotive company has three assembly lines (Line 1, Line 2, Line 3) to produce different car models. Each line has a series of stations, and each station takes a certain amount of time to complete its task. Additionally, there are transfer times between lines, and certain dependencies must be respected due to the sequential nature of some tasks. Your goal is to minimize the total production time by determining the optimal scheduling of tasks across these lines, considering the transfer times and dependencies.

```
1 lines = [
2     [5, 9, 3],
3     [6, 8, 4],
4     [7, 6, 5]
5 ]
6
7 transfer = [
8     [0, 2, 3],
9     [2, 0, 4],
10    [3, 4, 0]
11 ]
12
13 n = 3
14
15 dp = [[0] * n for _ in range(3)]
16
17 # First station
18 for i in range(3):
19     dp[i][0] = lines[i][0]
20
21 # Remaining stations
22 for j in range(1, n):
23     for i in range(3):
24         dp[i][j] = min(
25             dp[k][j - 1] + transfer[k][i]
26             for k in range(3)
27         ) + lines[i][j]
28
29 answer = min(dp[i][n - 1] for i in range(3))
30
31 print("Minimum production time:", answer)
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	9.052 Mb

Your Output

Minimum production time: 17

4. Write a c program to find the minimum path distance by using matrix form.

```
1 def tsp(graph):
2     n = len(graph)
3     visited = [False] * n
4     visited[0] = True
5
6     def solve(city, count, cost):
7         if count == n:
8             return cost + graph[city][0]
9
10        ans = float('inf')
11
12        for i in range(n):
13            if not visited[i]:
14                visited[i] = True
15                ans = min(ans, solve(i, count + 1,
16                                   cost + graph[city][i]))
17                visited[i] = False
18
19        return ans
20
21    return solve(0, 1, 0)
22
23
24 graph = [
25     [0, 10, 15, 20],
26     [10, 0, 35, 25],
27     [15, 35, 0, 30],
28     [20, 25, 30, 0]
29 ]
30
31 print("Minimum path distance:", tsp(graph))
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	8.956 Mb

Your Output

Minimum path distance: 80

5. Assume you are solving the Traveling Salesperson Problem for 4 cities (A, B, C, D) with known distances between each pair of cities. Now, you need to add a fifth city (E) to the problem.

```
1 from itertools import permutations
2
3 cities = ['A', 'B', 'C', 'D', 'E']
4
5 dist = {
6     'A': {'B':10, 'C':15, 'D':20, 'E':25},
7     'B': {'A':10, 'C':35, 'D':25, 'E':30},
8     'C': {'A':15, 'B':35, 'D':30, 'E':20},
9     'D': {'A':20, 'B':25, 'C':30, 'E':15},
10    'E': {'A':25, 'B':30, 'C':20, 'D':15}
11 }
12
13 start = 'A'
14 min_distance = float('inf')
15 best_route = None
16
17 for route in permutations(cities[1:]):
18     path = (start,) + route + (start,)
19     distance = 0
20
21     for i in range(len(path) - 1):
22         distance += dist[path[i]][path[i + 1]]
23
24     if distance < min_distance:
25         min_distance = distance
26         best_route = path
27
28 print("Shortest route:", " -> ".join(best_route))
29 print("Total distance:", min_distance)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	9.18 Mb

Your Output

Shortest route: A -> B -> D -> E -> C -> A
Total distance: 85

6. Given a string s, return the longest palindromic substring in S.

```
1 def longest_palindrome(s):
2     start = 0
3     max_len = 1
4
5     def expand(left, right):
6         while left >= 0 and right < len(s) and s[left] == s[right]:
7             left -= 1
8             right += 1
9         return left + 1, right - left - 1
10
11    for i in range(len(s)):
12        l1, len1 = expand(i, i)
13        l2, len2 = expand(i, i + 1)
14
15        if len1 > max_len:
16            start = l1
17            max_len = len1
18
19        if len2 > max_len:
20            start = l2
21            max_len = len2
22
23    return s[start:start + max_len]
24
25
26 print(longest_palindrome("babad"))
27 print(longest_palindrome("cbbd"))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0200 secs	8.636 Mb

Your Output

bab
bb

7. Given a string *s*, find the length of the longest substring without repeating characters.

```
1 def longest_substring(s):
2     seen = set()
3     left = 0
4     max_len = 0
5
6     for right in range(len(s)):
7         while s[right] in seen:
8             seen.remove(s[left])
9             left += 1
10
11         seen.add(s[right])
12         max_len = max(max_len, right - left + 1)
13
14     return max_len
15
16
17 print(longest_substring("abcabcbb"))
18 print(longest_substring("bbbbb"))
19 print(longest_substring("pwwkew"))
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	9.068 Mb

Your Output

```
3
1
3
```

8. Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words. Note that the same word in the dictionary may be reused multiple times in the segmentation.

```
1 def word_break(s, wordDict):
2     words = set(wordDict)
3     dp = [False] * (len(s) + 1)
4     dp[0] = True
5
6     for i in range(1, len(s) + 1):
7         for j in range(i):
8             if dp[j] and s[j:i] in words:
9                 dp[i] = True
10                break
11
12     return dp[len(s)]
13
14
15 print(word_break("leetcode", ["leet", "code"]))
16 print(word_break("applepenapple", ["apple", "pen"]))
17 print(word_break("catsandog", ["cats", "dog", "sand", "and", "cat"]))
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.024 Mb

Your Output

```
True
True
False
```

9. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. Consider the following dictionary { i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango }

```
1- def word_break(s, dictionary):
2-     dp = [False] * (len(s) + 1)
3-     dp[0] = True
4-
5-     for i in range(1, len(s) + 1):
6-         for word in dictionary:
7-             if i >= len(word) and dp[i - len(word)]:
8-                 if s[i - len(word):i] == word:
9-                     dp[i] = True
10-                    break
11-
12-     return dp[len(s)]
13-
14-
15- dictionary = {
16-     "i", "like", "sam", "sung", "samsung",
17-     "mobile", "ice", "cream", "icecream",
18-     "man", "go", "mango"
19- }
20-
21- print("ilike:", "Yes" if word_break("ilike", dictionary) else "No")
22- print("ilikesamsung:", "Yes" if word_break("ilikesamsung", dictionary) else "No")
```

Enter Input here

If your code takes input, add it in the above box before execution

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9.056 Mb

Your Output

ilike: Yes
ilikesamsung: Yes

10. Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified. You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly maxWidth characters. Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right. For the last line of text, it should be left-justified, and no extra space is inserted between words. A word is defined as a character sequence consisting of non-space characters only. Each word's length is guaranteed to be greater than 0 and not exceed maxWidth. The input array words contains at least one word.

```
1- def justify(words, maxWidth):
2-     result = []
3-     i = 0
4-
5-     while i < len(words):
6-         j = i
7-         length = 0
8-
9-         while j < len(words) and length + len(words[j]) + (j - i) <= maxWidth:
10-            length += len(words[j])
11-            j += 1
12-
13-         count = j - i
14-         spaces = maxWidth - length
15-
16-         if j == len(words) or count == 1:
17-             line = " ".join(words[i:j])
18-             line += " " * (maxWidth - len(line))
19-         else:
20-             gap = spaces // (count - 1)
21-             extra = spaces % (count - 1)
22-
23-             line = ""
24-
25-             for k in range(count - 1):
26-                 line += words[i + k]
27-                 line += " " * (gap + (1 if k < extra else 0))
28-
29-             line += words[j - 1]
30-
31-         result.append(line)
```

Enter Input here

If your code takes input, add it in the above box before execution

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.036 Mb

Your Output

This is an
example of text
justification.

11. Design a special dictionary that searches the words in it by a prefix and a suffix. Implement the WordFilter class: WordFilter(string[] words) Initializes the object with the words in the dictionary.f(string pref, string suff) Returns the index of the word in the dictionary, which has the prefix pref and the suffix suff. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.

```
1 class WordFilter:
2
3     def __init__(self, words):
4         self.words = words
5
6     def f(self, pref, suff):
7         for i in range(len(self.words) - 1, -1, -1):
8             word = self.words[i]
9
10            if word.startswith(pref) and word.endswith(suff):
11                return i
12
13        return -1
14
15 words = ["apple"]
16
17 obj = WordFilter(words)
18
19 print(obj.f("a", "e"))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.116 Mb

Your Output

0

12. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

```
1 INF = 99999
2
3 n = 4
4
5 graph = [
6     [0, 3, INF, -4],
7     [INF, 0, 4, 1],
8     [2, INF, 0, INF],
9     [INF, 6, -5, 0]
10 ]
11
12 print("Before Floyd's Algorithm:")
13 for row in graph:
14     print(row)
15
16 dist = [row[:] for row in graph]
17
18 for k in range(n):
19     for i in range(n):
20         for j in range(n):
21             dist[i][j] = min(
22                 dist[i][j],
23                 dist[i][k] + dist[k][j]
24             )
25
26 print("\nAfter Floyd's Algorithm:")
27 for row in dist:
28     print(row)
29
30 print("\nShortest distance from City 1 to City 3:", dist[0][2])
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.648 Mb

Your Output

Before Floyd's Algorithm:
[0, 3, 99999, -4]
[99999, 0, 4, 1]
[2, 99999, 0, 99999]
[99999, 6, -5, 0]

After Floyd's Algorithm:
[-7, -4, -9, -11]
[-2, 0, -4, -6]
[2, 0, 0, 0]

13. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure.

```
1 INF = 99999
2
3 n = 6
4
5 graph = [
6     [0, 1, 5, INF, INF, INF],
7     [1, 0, 2, 1, INF, INF],
8     [5, 2, 0, INF, 3, INF],
9     [INF, 1, INF, 0, 1, 6],
10    [INF, INF, 3, 1, 0, 2],
11    [INF, INF, INF, 6, 2, 0]
12 ]
13
14
15 def floyd(graph):
16     dist = [row[:] for row in graph]
17
18     for k in range(n):
19         for i in range(n):
20             for j in range(n):
21                 dist[i][j] = min(
22                     dist[i][j],
23                     dist[i][k] + dist[k][j]
24                 )
25
26     return dist
27
28
29 # Before link failure
30 dist = floyd(graph)
31
32 print("Before link failure:")
33 print("Router A to Router F =", dist[0][5])
34
35 # Remove B-D link
36 graph[1][3] = INF
37 graph[3][1] = INF
38
39 # After link failure
40 dist = floyd(graph)
41
42 print("After link failure:")
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 9.076 Mb

Your Output

Before link failure:
Router A to Router F = 5
After link failure:
Router A to Router F = 8

14. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

```
1 def find_city(n, edges, threshold):
2     INF = 99999
3
4     dist = [[INF] * n for _ in range(n)]
5
6     for i in range(n):
7         dist[i][i] = 0
8
9     for a, b, w in edges:
10        dist[a][b] = w
11        dist[b][a] = w
12
13
14     for k in range(n):
15         for i in range(n):
16             for j in range(n):
17                 dist[i][j] = min(
18                     dist[i][j],
19                     dist[i][k] + dist[k][j]
20                 )
21
22     answer = -1
23     minimum = INF
24
25     for i in range(n):
26         count = 0
27
28         for j in range(n):
29             if i != j and dist[i][j] <= threshold:
30                 count += 1
31
32         if count <= minimum:
33             minimum = count
34             answer = i
35
36     return answer
```

Enter Input here

If your code takes input, add it in the

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.232 Mb

Your Output

Answer: 0

15. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix.

```

1 def optimal_bst(keys, freq):
2     n = len(keys)
3
4     cost = [[0] * n for _ in range(n)]
5     root = [[0] * n for _ in range(n)]
6
7     for i in range(n):
8         cost[i][i] = freq[i]
9         root[i][i] = i
10
11     for length in range(2, n + 1):
12         for i in range(n - length + 1):
13             j = i + length - 1
14
15             total = sum(freq[i:j + 1])
16             cost[i][j] = float('inf')
17
18             for r in range(i, j + 1):
19                 left = cost[i][r - 1] if r > i else 0
20                 right = cost[r + 1][j] if r < j else 0
21
22                 value = left + right + total
23
24                 if value < cost[i][j]:
25                     cost[i][j] = value
26                     root[i][j] = r
27
28     print("Cost:", cost[0][n - 1])
29
30     print("\nCost Matrix:")
31     for row in cost:
32         print(row)
33
34     print("\nRoot Matrix:")
35     for row in root:
36         print(row)
37
38     return cost, root
39
40

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 9.344 Mb

Your Output

Cost Matrix:

```

[0.1, 0.4, 1.1, 1.7]
[0, 0.2, 0.8, 1.4]
[0, 0, 0.4, 1.0]
[0, 0, 0, 0.3]

```

Root Matrix:

```

[0, 1, 2, 2]
[0, 1, 2, 2]
[0, 0, 0, 2]
[0, 0, 0, 2]

```

16. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix.

```

2     n = len(keys)
3
4     cost = [[0] * n for _ in range(n)]
5     root = [[0] * n for _ in range(n)]
6
7     for i in range(n):
8         cost[i][i] = freq[i]
9         root[i][i] = i
10
11     for length in range(2, n + 1):
12         for i in range(n - length + 1):
13             j = i + length - 1
14
15             total = sum(freq[i:j + 1])
16             cost[i][j] = float('inf')
17
18             for r in range(i, j + 1):
19                 left = cost[i][r - 1] if r > i else 0
20                 right = cost[r + 1][j] if r < j else 0
21
22                 value = left + right + total
23
24                 if value < cost[i][j]:
25                     cost[i][j] = value
26                     root[i][j] = r
27
28     print("Minimum Cost:", cost[0][n - 1])
29
30     print("\nCost Matrix:")
31     for row in cost:
32         print(row)
33
34     print("\nRoot Matrix:")
35     for row in root:
36         print(row)
37
38     return cost, root
39
40

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.16 Mb

Your Output

Minimum Cost: 26

Cost Matrix:

```

[4, 8, 20, 26]
[0, 2, 10, 16]
[0, 0, 6, 12]
[0, 0, 0, 3]

```

Root Matrix:

```

[0, 1, 2, 3]
[0, 1, 2, 3]
[0, 0, 0, 2]
[0, 0, 0, 2]

```

17. A game on an undirected graph is played by two players, Mouse and Cat, who

alternate turns. The graph is given as follows: $graph[a]$ is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in $graph[1]$. Additionally, it is not allowed for the Cat to travel to the Hole (node 0). Then, the game can end in three ways:

- If ever the Cat occupies the same node as the Mouse, the Cat wins.
- If ever the Mouse reaches the Hole, the Mouse wins.
- If ever a position is repeated (i.e., the players are in the same position as a previous turn, and it is the same player's turn to move), the game is a draw.

Given a graph, and assuming both players play optimally, return

- 1 if the mouse wins the game,
- 2 if the cat wins the game, or
- 0 if the game is a draw.

```

23 q.append((0, c, 1))
24
25 # Cat catches mouse -> Cat wins
26 for i in range(1, n):
27     result[i][i][0] = 2
28     result[i][i][1] = 2
29     q.append((i, i, 0))
30     q.append((i, i, 1))
31
32 while q:
33     m, c, turn = q.popleft()
34     res = result[m][c][turn]
35
36     if turn == 0:
37         # Previous move was by Cat
38         parents = [(m, pc, 1) for pc in graph[c] if pc != 0]
39     else:
40         # Previous move was by Mouse
41         parents = [(pm, c, 0) for pm in graph[m]]
42
43     for pm, pc, pt in parents:
44         if result[pm][pc][pt] != 0:
45             continue
46
47         if (pt == 0 and res == 1) or (pt == 1 and res == 2):
48             result[pm][pc][pt] = res
49             q.append((pm, pc, pt))
50         else:
51             degree[pm][pc][pt] -= 1
52
53             if degree[pm][pc][pt] == 0:
54                 result[pm][pc][pt] = res
55                 q.append((pm, pc, pt))
56
57 return result[1][2][0]

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.724 Mb

Your Output

Result: 0

18. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $edges[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $succProb[i]$. Given two nodes $start$ and end , find the path with the maximum probability of success to go from $start$ to end and return its success probability. If there is no path from $start$ to end , return 0. Your answer will be accepted if it differs from the correct answer by at most $1e-5$.

```

1 import heapq
2
3 def max_probability(n, edges, prob, start, end):
4     graph = [[] for _ in range(n)]
5
6     for i in range(len(edges)):
7         a, b = edges[i]
8         graph[a].append((b, prob[i]))
9         graph[b].append((a, prob[i]))
10
11     best = [0] * n
12     best[start] = 1
13
14     pq = [(-1, start)]
15
16     while pq:
17         probability, node = heapq.heappop(pq)
18         probability = -probability
19
20         if node == end:
21             return probability
22
23         for next_node, p in graph[node]:
24             new_prob = probability * p
25
26             if new_prob > best[next_node]:
27                 best[next_node] = new_prob
28                 heapq.heappush(pq, (-new_prob, next_node))
29
30     return 0

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.38 Mb

Your Output

Maximum probability: 0.25

19. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., $\text{grid}[0][0]$). The robot tries to move to the bottom-right corner (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

```

1 def unique_paths(m, n):
2     dp = [[0] * n for _ in range(m)]
3
4     for i in range(m):
5         dp[i][0] = 1
6
7     for j in range(n):
8         dp[0][j] = 1
9
10    for i in range(1, m):
11        for j in range(1, n):
12            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
13
14    return dp[m - 1][n - 1]
15
16
17 print("Test Case 1:", unique_paths(3, 7))
18 print("Test Case 2:", unique_paths(3, 2))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.972 Mb

Your Output

Test Case 1: 28
Test Case 2: 3

20. Given an array of integers `nums`, return the number of good pairs. A pair (i, j) is called good if $\text{nums}[i] == \text{nums}[j]$ and $i < j$.

```

1- def good_pairs(nums):
2-     count = 0
3-
4-     for i in range(len(nums)):
5-         for j in range(i + 1, len(nums)):
6-             if nums[i] == nums[j]:
7-                 count += 1
8-
9-     return count
10
11
12 print(good_pairs([1, 2, 3, 1, 1, 3]))
13 print(good_pairs([1, 1, 1, 1]))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 8.992 Mb

Your Output

4
6

21. There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

```

4- dist = [[INF] * n for _ in range(n)]
5-
6- for i in range(n):
7-     dist[i][i] = 0
8-
9- for a, b, w in edges:
10-     dist[a][b] = w
11-     dist[b][a] = w
12
13 # Floyd-Warshall Algorithm
14 for k in range(n):
15     for i in range(n):
16         for j in range(n):
17             dist[i][j] = min(
18                 dist[i][j],
19                 dist[i][k] + dist[k][j]
20             )
21
22 answer = -1
23 minimum = INF
24
25 for i in range(n):
26     count = 0
27
28     for j in range(n):
29         if i != j and dist[i][j] <= threshold:
30             count += 1
31
32     if count <= minimum:
33         minimum = count
34         answer = i
35
36

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.196 Mb

Your Output

City: 3

22. You are given a network of n nodes, labeled from 1 to n . You are also given `times`, a list of travel times as directed edges `times[i] = (ui, vi, wi)`, where `ui` is the source node, `vi` is the target node, and `wi` is the time it takes for a signal to travel from source to target. We will send a signal from a given node `k`. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

```

3 def network_delay_time(times, n, k):
4     graph = [[] for _ in range(n + 1)]
5
6     for u, v, w in times:
7         graph[u].append((v, w))
8
9     distance = [float('inf')] * (n + 1)
10    distance[k] = 0
11
12    pq = [(0, k)]
13
14    while pq:
15        time, node = heapq.heappop(pq)
16
17        if time > distance[node]:
18            continue
19
20        for next_node, weight in graph[node]:
21            new_time = time + weight
22
23            if new_time < distance[next_node]:
24                distance[next_node] = new_time
25                heapq.heappush(pq, (new_time, next_node))
26
27    answer = max(distance[1:])
28
29    if answer == float('inf'):
30        return -1
31
32    return answer
33
34
35 print(network_delay_time(
36     [[2, 1, 1], [2, 3, 1], [3, 4, 1]],
37     4,
38     2
39 ))

```

Enter Input here

If your code takes input, add it in the above

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.116 Mb

Your Output

```

2
1
-1

```